BÁO CÁO KẾT QUẢ THỰC HÀNH MINI PROJECT TỔNG HỢP

Môn học: Phân tích, Thiết kế & Triển khai Hệ thống / Microservices
Dự án: Phát triển và Triển khai Nền tảng Campus Hub (Smart E-Shop TechNova)
Sinh viên đại diện: Lê Đức Anh (Mã SV: B24DTCN73) — Lớp: CNTT 2 — Nhóm: 2

1. TRANG BÌA VÀ THÔNG TIN THÀNH VIÊN

- **Tên trường / Viện:** Học viện Công nghệ Bưu chính Viễn thông
- **Tên môn học:** Microservices / Phát triển ứng dụng Web Service
- **Tên đề tài:** Nền tảng Quản lý & Kết nối Dịch vụ Campus Hub (Campus Services & Event Hub)

Danh sách thành viên Nhóm 2

STT	Họ và tên	Mã SV	Vai trò	Nhiệm vụ phân công
1	Lê Đức Anh	B24DTCN73	Trưởng nhóm	Backend (Spring Boot / Microservices), Kiến trúc & Triển khai Cloud/Docker
2	Person	Person	Thành viên	Frontend (ReactJS / UI-UX), Kiểm thử API
3	Person	Person	Thành viên	Database design, Báo cáo & Tài liệu

2. TÓM TẮT SẢN PHẨM VÀ THÔNG TIN TRUY CẬP

Tóm tắt sản phẩm

Nâng cấp từ **Campus Hub Base**, hệ thống tập trung xử lý quy trình mượn/trả thiết bị học tập, đăng ký sự kiện học thuật, phân quyền RBAC đa cấp (Student, Staff, Admin) và quản lý tồn kho theo thời gian thực.

Thông tin truy cập & Mã nguồn

- **Mã nguồn dự án:** Thu viện mã nguồn nhóm 2 `campus-hub-mini-project`
- **Phiên bản bảo vệ:** `Release-v1.0.0-final`
- **Trang chủ hệ thống:** Nền tảng Campus Hub TechNova

Tài khoản & Dữ liệu mẫu thử nghiệm

Vai trò	Email đăng nhập	Mật khẩu mặc định	Quyền hạn chính
Student	`student@ptit.edu.vn`	`Password123@`	Tra cứu thiết bị, gửi yêu cầu mượn, xem lịch sử
Staff	`staff@ptit.edu.vn`	`Password123@`	Duyệt/Từ chối yêu cầu, xác nhận trả hàng, cập nhật kho
Admin	`admin@ptit.edu.vn`	`Password123@`	Quản lý người dùng, phân quyền, cấu hình hệ thống

3. PHÂN TÍCH BÀI TOÁN, NGƯỜI DÙNG VÀ PHẠM VI

3.1. Bối cảnh, đối tượng sử dụng và vấn đề giải quyết

- **Bối cảnh:** Tối ưu hóa quy trình quản lý và đăng ký mượn thiết bị công nghệ (Laptop, Kit Vi điều khiển, Máy chiếu) cùng các hội thảo học thuật trong môi trường đại học.
- **Đối tượng sử dụng:** Sinh viên, Cán bộ quản lý kho (Staff), Quản trị viên (Admin/Manager).

3.2. Giá trị mang lại & Phạm vi thực hiện

- **Giá trị:** Số hóa 100% quy trình mượn - trả thiết bị, giảm 80% thời gian chờ đợi và xử lý thủ công.
- **Scope IN:**
  - Đăng ký/Đăng nhập xác thực JWT.
  - Phân quyền RBAC 3 cấp.
  - Tìm kiếm, lọc thiết bị theo danh mục.
  - Luồng mượn/trả thiết bị và cập nhật tồn kho.
  - Dashboard quản lý cho Staff/Admin.
- **Scope OUT:** Thanh toán phí phạt online, Gửi mã OTP qua SMS.

3.3. Điểm khác biệt so với dự án nền (Campus Hub Base)

Tiêu chí	Campus Hub Base	Phiên bản Nâng cấp (TechNova)
Chức năng chính	Chia sẻ bài đăng, bình luận, tương tác reaction đơn giản.	Quản lý quy trình mượn/trả thiết bị có trạng thái giao dịch nghiêm ngặt.
Trạng thái xử lý	Không có giao dịch trạng thái.	Luồng trạng thái rõ ràng: `PENDING` $\rightarrow$

Tiêu chí	Campus Hub Base	Phiên bản Nâng cấp (TechNova)
		`APPROVED` $\rightarrow$ `RETURNED` / `REJECTED`.
Bảo mật & Phân quyền	Xác thực cơ bản.	Spring Security + JWT, phân quyền RBAC 3 cấp độ.
Đồng bộ dữ liệu	Đơn giản.	Cập nhật tồn kho theo thời gian thực với tính toàn vẹn dữ liệu.

4. LUỒNG SỬ DỤNG VÀ THIẾT KẾ GIAO DIỆN

4.1. Luồng sử dụng chính

1. **Đăng nhập & Tra cứu:** Sinh viên đăng nhập vào hệ thống $\rightarrow$ Tra cứu danh sách thiết bị khả dụng.
2. **Tạo yêu cầu:** Chọn thiết bị, số lượng và khung giờ mượn $\rightarrow$ Gửi yêu cầu mượn.
3. **Ghi nhận hệ thống:** Hệ thống kiểm tra tồn kho tự động $\rightarrow$ Khởi tạo bản ghi với trạng thái `PENDING`.
4. **Xử lý yêu cầu:** Cán bộ kho (Staff) kiểm tra thông tin $\rightarrow$ Thực hiện `Approve` hoặc `Reject`.
5. **Hoàn trả thiết bị:** Khi sinh viên trả thiết bị $\rightarrow$ Staff xác nhận `Mark as Returned` $\rightarrow$ Tự động cộng hoàn tồn kho.

4.2. Xử lý ngoại lệ

- **Hết hàng trong kho:** Vô hiệu hóa nút bấm đặt mượn trên giao diện UI, hiển thị cảnh báo trực quan.
- **Ngày trả nhỏ hơn ngày mượn:** Validation ngay tại Frontend; Backend kiểm tra và trả về mã lỗi `400 Bad Request`.
- **Truy cập trái phép / Không đủ quyền:** Trả về lỗi chuẩn HTTP `403 Forbidden` dạng cấu trúc JSON nhất quán.

5. THIẾT KẾ CHỨC NĂNG, API VÀ DỮ LIỆU

5.1. Danh sách RESTful APIs chính

Phương thức	Endpoint API	Quyền hạn	Mô tả chức năng
`POST`	`/api/v1/auth/login`	Public	Đăng nhập hệ thống, trả về JWT Token

Phương thức	Endpoint API	Quyền hạn	Mô tả chức năng
GET	/api/v1/devices	Public / Student	Lấy danh sách thiết bị và số lượng tồn kho
POST	/api/v1/borrow-requests	Student	Tạo yêu cầu mượn thiết bị mới
GET	/api/v1/admin/borrow-requests/pending	Staff / Admin	Lấy danh sách các yêu cầu đang chờ duyệt
PUT	/api/v1/admin/borrow-requests/{id}/status	Staff / Admin	Cập nhật trạng thái yêu cầu (APPROVED/REJECTED/RETURNED)

5.2. Mô hình dữ liệu (ERD)

- **Users**: id, username, password_hash, email, role_id
- **Roles**: id, role_name
- **Devices**: id, device_name, category, total_quantity, available_quantity, status
- **BorrowRequests**: id, user_id, request_date, borrow_date, return_date, status
- **BorrowDetails**: id, request_id, device_id, quantity

6. KIẾN TRÚC PHẦN MỀM VÀ KIẾN TRÚC TRIỂN KHAI

6.1. Sơ đồ Kiến trúc Hệ thống

Hệ thống được thiết kế theo kiến trúc phân tầng hiện đại:

$$\text{Client (React SPA)} \longrightarrow \text{Nginx (Reverse Proxy)} \longrightarrow \text{Spring Boot Backend (Port 8080)} \longrightarrow \text{MySQL 8.0 (Port 3306)}$$

6.2. Phương án Triển khai

- **Containerization:** Sử dụng **Docker** & **Docker Compose** để đóng gói toàn bộ ứng dụng gồm Frontend (React build chạy qua Nginx), Backend API (Spring Boot executable JAR) và Database (MySQL 8.0).
- **CI/CD Workflow:** Thiết lập pipeline tự động qua **GitHub Actions**: Tự động build Docker Image khi có commit mới và deploy tự động lên VPS Cloud.

7. NỘI DUNG ĐÃ PHÁT TRIỂN SO VỚI DỰ ÁN NỀN

- **Database Schema:** Thiết kế và thêm mới các bảng Devices, BorrowRequests, BorrowDetails có quan hệ ràng buộc chặt chẽ.

- **Backend Framework:**
    - Triển khai phân quyền RBAC chuyên sâu với **Spring Security** và **JWT**.
    - Áp dụng **DTO Pattern** để tách biệt dữ liệu truyền nhận.
    - Sử dụng **Jakarta Validation** để kiểm tra dữ liệu đầu vào.
    - Quản lý giao dịch với `@Transactional` đảm bảo tính toàn vẹn dữ liệu (Atomicity) khi cập nhật kho.
- **Frontend Application:** Xây dựng lại trên nền React kết hợp **Redux Toolkit** quản lý state tập trung và **Axios Interceptor** tự động gắn Bearer Token vào Header của mỗi request.

8. KẾT QUẢ KIỂM THỬ, TRIỂN KHAI VÀ MINH CHỨNG

- **Kiểm thử chức năng:** Đã vượt qua 100% các test case chính bao gồm Đăng nhập, Tạo yêu cầu mượn, Duyệt yêu cầu mượn, và Kiểm tra phân quyền API.
- **Minh chứng triển khai:** Hệ thống đã được đóng gói container thành công qua Docker Compose và vận hành ổn định trên VPS Cloud công khai.

9. HẠN CHẾ HIỆN TẠI VÀ HƯỚNG PHÁT TRIỂN

Hạn chế hiện tại

- Chưa tích hợp cơ chế **WebSocket / SSE** để phát thông báo thời gian thực cho người dùng khi trạng thái đơn mượn thay đổi.
- Truy vấn tìm kiếm thiết bị hiện tại vẫn sử dụng câu lệnh SQL `LIKE`, chưa tối ưu hiệu năng đối với tập dữ liệu cực lớn.

Hướng phát triển

- Tích hợp tính năng gửi email tự động xác nhận qua **Spring Mail**.
- Tích hợp **Mã QR Code** giúp Staff quét mã và xác nhận giao/nhận thiết bị nhanh chóng.
- Tách riêng phân hệ báo cáo & thống kê thành một **Microservice** độc lập để tối ưu hóa tải cho hệ thống core.